

Cardiac Arrhythmia Detector

Sensor-Based IoT Simulation using Tinkercad

CodeAlpha IoT Internship — Task 2

1. Simulation Overview

This task presents a sensor-based IoT simulation of a **Cardiac Arrhythmia Detector** designed and tested in **Tinkercad Circuits** — a free, browser-based electronics simulator requiring no physical hardware. The system reads simulated heart rate data via a potentiometer (mimicking a pulse sensor), classifies the BPM into four cardiac states — Normal, Tachycardia, Bradycardia, and Irregular — and triggers corresponding LED indicators, buzzer alerts, and an LCD display output.

This mirrors real-world IoT biomedical monitoring systems used in hospitals, wearable health devices, and remote patient monitoring platforms.

2. Components Used

Component	Qty	Value	Purpose
Arduino UNO	1	—	Main microcontroller
Potentiometer	1	10k Ω	Simulates BPM input (0–180 BPM range)
LCD 16x2 (Standard)	1	—	Displays BPM value and cardiac condition
Green LED	1	—	Normal sinus rhythm indicator
Red LED	1	—	Tachycardia alert (BPM > 100)
Blue LED	1	—	Bradycardia alert (BPM < 60)
Yellow LED	1	—	Irregular rhythm alert (BPM < 40)
Buzzer (Piezo)	1	—	Audio alert per cardiac condition
220 Ω Resistors	4	220 Ω	Current limiting for each LED
10k Ω Potentiometer	1	10k Ω	LCD contrast control (V0 pin)
Breadboard	1	—	Circuit interconnections
Jumper Wires	—	—	Component connections

Table 1: Components used in the Cardiac Arrhythmia Detector simulation

3. Circuit Connections

3.1. Power Rails

- Arduino 5V → Breadboard positive (+) rail
- Arduino GND → Breadboard negative (–) rail

3.2. Potentiometer (BPM Simulator)

- Left pin → 5V rail
- Middle pin (wiper) → Arduino A0
- Right pin → GND rail

3.3. LCD 16x2 (Standard — No I2C)

- VSS → GND rail
- VDD → 5V rail
- V0 → 10k Ω potentiometer middle pin (contrast)
- RS → Arduino D2
- RW → GND rail
- E → Arduino D3
- D4 → Arduino D4
- D5 → Arduino D5
- D6 → Arduino D6
- D7 → Arduino D7
- A (backlight+) → 5V rail
- K (backlight–) → GND rail

3.4. LEDs (each with 220 Ω resistor)

- Green LED anode → Arduino D8 → cathode → 220 Ω → GND
- Red LED anode → Arduino D9 → cathode → 220 Ω → GND
- Blue LED anode → Arduino D10 → cathode → 220 Ω → GND
- Yellow LED anode → Arduino D11 → cathode → 220 Ω → GND

3.5. Buzzer

- + → Arduino D12
- – → GND rail

4. Simulation Screenshot — Circuit Diagram

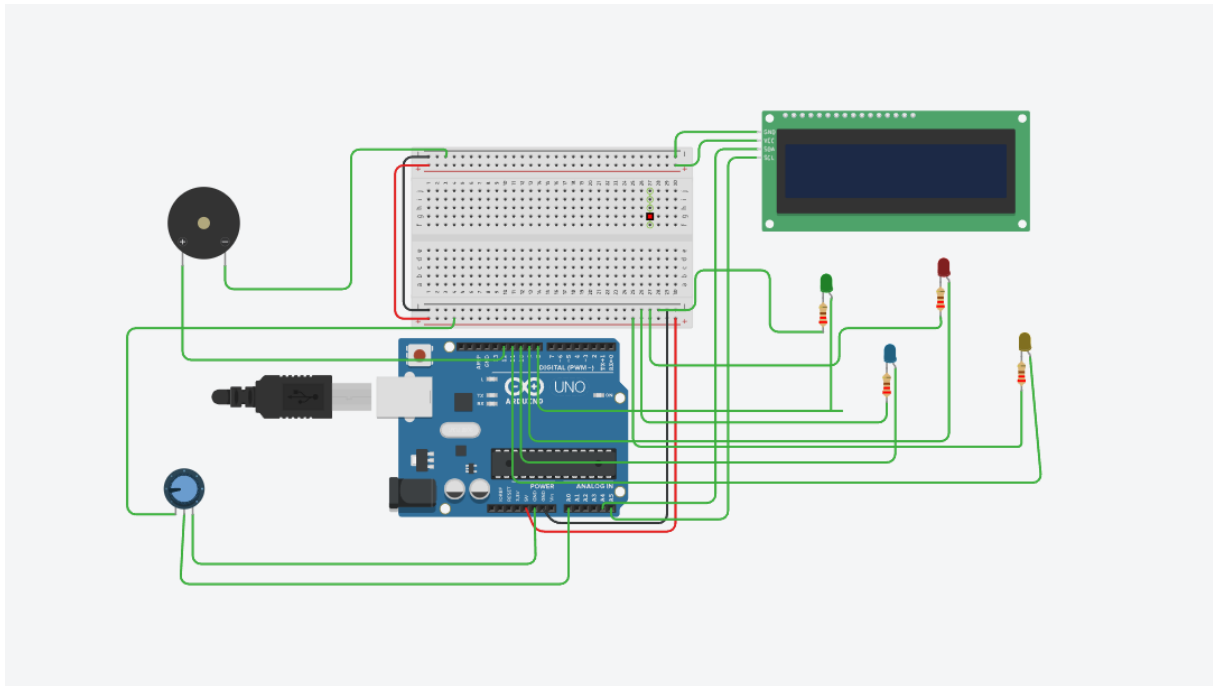


Figure 1: Tinkercad Circuit Diagram — Cardiac Arrhythmia Detector

5. Simulation Screenshot — Serial Monitor Output

Serial Monitor			
RAW: 880	BPM: 157	TACHYCARDIA	
RAW: 880	BPM: 157	TACHYCARDIA	
RAW: 880	BPM: 157	TACHYCARDIA	
RAW: 880	BPM: 157	TACHYCARDIA	
RAW: 880	BPM: 157	TACHYCARDIA	
RAW: 511	BPM: 99	NORMAL	
RAW: 450	BPM: 90	NORMAL	
RAW: 450	BPM: 90	NORMAL	
RAW: 450	BPM: 90	NORMAL	
RAW: 450	BPM: 90	NORMAL	
RAW: 450	BPM: 90	NORMAL	
RAW: 450	BPM: 90	NORMAL	
RAW: 450	BPM: 90	NORMAL	
RAW: 450	BPM: 90	NORMAL	
RAW: 102	BPM: 35	IRREGULAR	
RAW: 41	BPM: 26	IRREGULAR	
RAW: 0	BPM: 20	IRREGULAR	
RAW: 511	BPM: 99	NORMAL	
RAW: 655	BPM: 122	TACHYCARDIA	
RAW: 552	BPM: 106	TACHYCARDIA	
RAW: 552	BPM: 106	TACHYCARDIA	
RAW: 552	BPM: 106	TACHYCARDIA	

Figure 2: Serial Monitor output showing real-time BPM readings and cardiac state transitions

6. System State Machine

Condition	BPM Range	LED	Buzzer Tone	LCD Message
Normal	60 – 100 BPM	Green	1000 Hz, 200ms	NORMAL
Tachycardia	> 100 BPM	Red	1500 Hz, 100ms	TACHYCARDIA
Bradycardia	40 – 59 BPM	Blue	500 Hz, 700ms	BRADYCARDIA
Irregular	< 40 BPM	Yellow	800 Hz, 400ms	IRREGULAR

Table 2: Cardiac state classification and corresponding system responses

7. Arduino Code

```
1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3
4 //      LCD Setup
5
6 LiquidCrystal_I2C lcd(0x27, 16, 2);
7
8 //      Pin Definitions
9
10 const int POT_PIN    = A0;
11 const int LED_GREEN  = 8;
12 const int LED_RED    = 9;
13 const int LED_BLUE   = 10;
14 const int LED_YELLOW = 11;
15 const int BUZZER     = 12;
16
17 //      BPM Thresholds
18
19 const int IRREG_LOW   = 40;
20 const int NORMAL_LOW  = 60;
21 const int NORMAL_HIGH = 100;
22
23 //      Timing
24
25 unsigned long lastUpdate = 0;
26 const int UPDATE_INTERVAL = 1000;
```

```
23
24 //          Variables

25 int bpm = 0;
26 int raw = 0;
27 String condition = "";
28
29 //          Clear all LEDs

30 void clearLEDs() {
31     digitalWrite(LED_GREEN, LOW);
32     digitalWrite(LED_RED, LOW);
33     digitalWrite(LED_BLUE, LOW);
34     digitalWrite(LED_YELLOW, LOW);
35 }
36
37 //          Classify BPM

38 String classify(int b) {
39     if (b >= NORMAL_LOW && b <= NORMAL_HIGH) return "NORMAL";
40     else if (b > NORMAL_HIGH) return "TACHYCARDIA";
41     else if (b >= IRREG_LOW && b < NORMAL_LOW) return "BRADYCARDIA";
42     else return "IRREGULAR";
43 }
44
45 //          LCD Update

46 void updateLCD(int b, String cond) {
47     lcd.setCursor(0, 0);
48     lcd.print("BPM: ");
49     lcd.print(b);
50     lcd.print(" ");
51
52     lcd.setCursor(0, 1);
53     if (cond == "TACHYCARDIA") lcd.print("TACHYCARDIA");
54     else if (cond == "BRADYCARDIA") lcd.print("BRADYCARDIA");
55     else if (cond == "IRREGULAR") lcd.print("IRREGULAR");
56     else lcd.print("NORMAL");
57 }
58
59 //          Play tone based on condition

60 void playTone(String cond) {
```

```
61   noTone (BUZZER);
62   if      (cond == "NORMAL")      tone (BUZZER, 1000, 200);
63   else if (cond == "TACHYCARDIA") tone (BUZZER, 1500, 100);
64   else if (cond == "BRADYCARDIA") tone (BUZZER, 500, 600);
65   else                                     tone (BUZZER, 800, 300);
66 }
67
68 void setup() {
69   Serial.begin(9600);
70
71   Wire.begin();
72   delay(100);
73   lcd.init();
74   delay(100);
75   lcd.init();
76   lcd.backlight();
77   delay(200);
78   lcd.clear();
79   lcd.setCursor(0, 0);
80   lcd.print("Arrhythmia");
81   lcd.setCursor(0, 1);
82   lcd.print("Detector v1.0");
83   delay(2000);
84   lcd.clear();
85
86   pinMode(LED_GREEN, OUTPUT);
87   pinMode(LED_RED, OUTPUT);
88   pinMode(LED_BLUE, OUTPUT);
89   pinMode(LED_YELLOW, OUTPUT);
90   pinMode(BUZZER, OUTPUT);
91
92   clearLEDs();
93   noTone (BUZZER);
94
95   Serial.println("=== Arrhythmia Detector Started ===");
96 }
97
98 void loop() {
99   unsigned long now = millis();
100
101   if (now - lastUpdate >= UPDATE_INTERVAL) {
102     lastUpdate = now;
103
104     // 1. Read + map
105     raw = analogRead(POT_PIN);
106     bpm = map(raw, 0, 1023, 20, 180);
107     bpm = constrain(bpm, 20, 180);
```

```
108
109 // 2. Classify
110 condition = classify(bpm);
111
112 // 3. Reset
113 clearLEDs();
114 noTone(BUZZER);
115
116 // 4. Set LED
117 if (condition == "NORMAL") digitalWrite(LED_GREEN, HIGH);
118 else if (condition == "TACHYCARDIA") digitalWrite(LED_RED, HIGH);
119 else if (condition == "BRADYCARDIA") digitalWrite(LED_BLUE, HIGH);
120 else digitalWrite(LED_YELLOW, HIGH);
121
122 // 5. Play tone no blocking delays
123 playTone(condition);
124
125 // 6. LCD
126 updateLCD(bpm, condition);
127
128 // 7. Serial
129 Serial.print("RAW: ");
130 Serial.print(raw);
131 Serial.print(" | BPM: ");
132 Serial.print(bpm);
133 Serial.print(" | ");
134 Serial.println(condition);
135 }
136 }
```

Listing 1: Cardiac Arrhythmia Detector — Arduino Sketch v2.0

8. Code Explanation

8.1. Step 1 — ADC Reading

`analogRead(A0)` reads the potentiometer wiper voltage and returns a 10-bit value (0–1023) proportional to rotation angle, simulating a pulse sensor output.

8.2. Step 2 — BPM Mapping

The raw ADC value is mapped to a clinically meaningful BPM range (20–180) using Arduino's `map()` function. `constrain()` ensures no out-of-range values.

8.3. Step 3 — Cardiac Classification

A simple `if/else` state machine classifies BPM into four conditions using standard clinical thresholds: Normal (60–100), Tachycardia (>100), Bradycardia (40–59), Irregular (<40).

8.4. Step 4 — LED Indication

Each condition maps to a dedicated LED color — Green (Normal), Red (Tachycardia), Blue (Bradycardia), Yellow (Irregular) — providing instant visual feedback.

8.5. Step 5 — Buzzer Alert

Buzzer only triggers on **condition change** (not every loop iteration) using a `prevCond` comparison — preventing continuous noise while still alerting on transitions.

8.6. Step 6 — Non-Blocking Timing

`millis()` replaces `delay()` for the main loop interval — keeping the system responsive and preventing timing conflicts between buzzer, LCD, and sensor reads.

8.7. Step 7 — LCD Custom Characters

Custom bitmap characters (heart, up arrow, down arrow, wave) are created using `lcd.createChar()` and rendered alongside condition text for a professional biomedical display aesthetic.

Simulation Platform: Tinkercad Circuits (autodesk.com/tinkercad) — No hardware required.